

Ejercicios de sistemas basados en agentes

Big Data Processing II

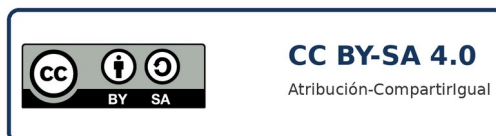
Máster Universitario en Análisis de Datos Deportivos

Curso 2026-2027

Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández



Depósito institucional BURJC Digital

<https://burjcdigital.urjc.es>

Información de licencia y reutilización

Material docente en abierto de la Universidad Rey Juan Carlos.

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández.
Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons
Atribución-CompartirIgual 4.0 Internacional, disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

La licencia se aplica al contenido original de la obra. Los logotipos, marcas, citas y
materiales de terceros conservan su régimen jurídico propio y quedan excluidos de la
licencia, salvo indicación expresa.

Agentes aplicados al análisis deportivo

Se plantean diferentes ejercicios basados en agentes LLM + Tools + Skills (framework práctico: LangGraph).

Objetivo: diseñar flujos agénticos controlados e integrar herramientas externas.

1) Bloque teórico (sin código)

Ejercicio T1 — Tool vs Skill vs Agent (10–12 líneas)

Define con tus palabras (sin copiar definiciones) las diferencias entre tool, skill y agente LLM. Incluye un ejemplo de cada uno en el contexto de un asistente universitario (normativa, tutorías, evaluación, etc.).

Ejercicio T2 — Diseño de control: evitar loops y alucinaciones (12–15 líneas)

Explica por qué los agentes pueden entrar en bucles o tomar acciones incorrectas. Propón mecanismos concretos de control.

Ejercicio T3 — RAG como tool dentro de un agente (8–10 líneas)

Describe cómo integrarías un sistema RAG como tool dentro de un agente. ¿Qué decide el agente? ¿Qué devuelve la tool? ¿Qué riesgos aparecen (p. ej., retrieval irrelevante) y cómo los mitigarías?

2) Bloque práctico con LangGraph

Entrega recomendada: un repositorio con scripts o notebooks por ejercicio, y un README con instrucciones. En cada ejercicio, incluye un ejemplo de ejecución y una breve discusión (5–10 líneas) de lo observado.

Ejercicio P1 — Flujo sencillo (grafo lineal con estado)

Implementa un flujo en LangGraph con 3 nodos y estado compartido:

- 1) Nodo "preprocess": normaliza la pregunta (p. ej., minúsculas, recorta espacios, detecta idioma simple).

- 2) Nodo "llm_answer": el LLM responde con un formato fijo (título + 3 bullets).

- 3) Nodo "postprocess": añade una sección final "Calidad" con dos checks: "¿hay respuesta?" y "¿es clara?".

Requisitos:

- Define un estado TypedDict con campos: question (str), normalized_question (str), answer (str).
- El grafo debe compilar y ejecutarse con `app.invoke({...})`.



- Añade logging simple para ver el paso por nodos.

Pista: este ejercicio busca que entiendas el concepto de state y cómo LangGraph estructura un pipeline, sin herramientas todavía.

Ejercicio P2 (opcional) — Agente con tool RAG (conocimiento externo)

Integra una tool tipo RAG como nodo de LangGraph (puedes reutilizar tu vector store de ejercicios de RAG). El agente debe:

- 1) Detectar si la pregunta requiere conocimiento documental.
- 2) Si sí, llamar a "rag_tool" (retrieval) y luego responder con groundedness.
- 3) Si no, responder directamente.

Requisitos:

- El estado debe incluir: question, decision, retrieved_context, sources, answer.
- La respuesta debe terminar con "Fuentes:" listando metadatos.
- Debe existir una política de abstención: si no hay contexto suficiente, decir "No puedo responder con la información disponible."

3) Entregables y criterios de evaluación

Entregables mínimos:

- Código ejecutable (scripts o notebooks) por ejercicio práctico.
- README con instalación y ejecución
- Capturas o logs de ejecuciones representativas
- Respuestas a T1–T3

Anexo — Esqueleto mínimo de LangGraph (referencia)

```
from typing import TypedDict
from langgraph.graph import StateGraph, END
```

```
class State(TypedDict):
    question: str
    answer: str
```

```
def node_a(state: State) -> State:
    # ...
    return state
```

```
graph = StateGraph(State)
graph.add_node("a", node_a)
graph.set_entry_point("a")
graph.add_edge("a", END)
```

```
app = graph.compile()  
print(app.invoke({"question": "Hola", "answer": ""}))
```

MATERIAL DOCENTE EN ABIERTO